```
"""
FreeAlexya : serveur MCP 100 % local, sans crédits, branché sur ComfyUI.
Outils : generate_image, generate_video, check_generation,
         upload_reference, get_account, list_generations
"""
import json, os, random, shutil, sqlite3, subprocess, threading, time, uuid
from pathlib import Path

import requests
from mcp.server.fastmcp import FastMCP
from starlette.responses import FileResponse, PlainTextResponse

COMFY = os.getenv("COMFY_URL", "http://127.0.0.1:8188")
PUBLIC = os.getenv("PUBLIC_URL", "http://127.0.0.1:8000")  # URL publique (tunnel
PORT = int(os.getenv("PORT", "8000"))
MODEL = os.getenv("MODEL", "sdxl_turbo")  # "sdxl_turbo" (GPU 8-16 Go, Colab/Kagg
IMAGE_CKPT = os.getenv("IMAGE_CKPT", {
    "sdxl_turbo": "sd_xl_turbo_1.0_fp16.safetensors",
    "flux": "flux1-schnell-fp8.safetensors"}[MODEL])
DATA = Path("data"); OUT = DATA / "out"; REF = DATA / "ref"
for d in (OUT, REF):
    d.mkdir(parents=True, exist_ok=True)

# ---------- base de jobs ----------
db = sqlite3.connect(DATA / "jobs.db", check_same_thread=False)
db.execute("""CREATE TABLE IF NOT EXISTS jobs(
  id TEXT PRIMARY KEY, kind TEXT, prompt TEXT, params TEXT,
  status TEXT, output_url TEXT, error TEXT, created REAL)""")
lock = threading.Lock()

def set_job(jid, **kw):
    with lock:
        for k, v in kw.items():
            db.execute(f"UPDATE jobs SET {k}=? WHERE id=?", (v, jid))
        db.commit()

# ---------- workflows ComfyUI ----------
SIZES = {"1:1": (1024, 1024), "16:9": (1344, 768), "9:16": (768, 1344),
         "4:3": (1152, 864), "3:4": (864, 1152)}

def image_workflow(prompt, aspect, seed):
    w, h = SIZES.get(aspect, (1024, 1024))
    if MODEL == "sdxl_turbo":  # résolution native ~512 : plus rapide, qualité co
```

```
        w, h = (w // 64 * 64 * 3 // 4) // 64 * 64, (h // 64 * 64 * 3 // 4) // 64
        sampler, steps, latent = "euler_ancestral", 4, "EmptyLatentImage"
    else:
        sampler, steps, latent = "euler", 4, "EmptySD3LatentImage"
    return {
        "1": {"class_type": "CheckpointLoaderSimple", "inputs": {"ckpt_name": IMA
        "2": {"class_type": "CLIPTextEncode", "inputs": {"text": prompt, "clip":
        "3": {"class_type": "CLIPTextEncode", "inputs": {"text": "", "clip": ["1"
        "4": {"class_type": latent, "inputs": {"width": w, "height": h, "batch_si
        "5": {"class_type": "KSampler", "inputs": {
            "model": ["1", 0], "positive": ["2", 0], "negative": ["3", 0],
            "latent_image": ["4", 0], "seed": seed, "steps": steps, "cfg": 1.0,
            "sampler_name": sampler, "scheduler": "simple", "denoise": 1.0}},
        "6": {"class_type": "VAEDecode", "inputs": {"samples": ["5", 0], "vae": [
        "7": {"class_type": "SaveImage", "inputs": {"images": ["6", 0], "filename
    }

VIDEO_SIZES = {"16:9": (576, 320), "9:16": (320, 576), "1:1": (384, 384),
               "4:3": (512, 384), "3:4": (384, 512)}
LTX_CKPT = os.getenv("LTX_CKPT", "ltx-video-2b-v0.9.5.safetensors")
LTX_T5 = os.getenv("LTX_T5", "t5xxl_fp8_e4m3fn_scaled.safetensors")
FPS = 24

def video_workflow(prompt, aspect, seed, frames):
    """Si workflows/video_api.json existe (export ComfyUI, placeholders {{prompt}
    {{width}} {{height}} {{length}}), il est utilisé. Sinon : LTX-Video texte->vi
    w, h = VIDEO_SIZES.get(aspect, (576, 320))
    tpl = Path("workflows/video_api.json")
    if tpl.exists():
        txt = tpl.read_text()
        for k, v in {"{{prompt}}": json.dumps(prompt)[1:-1], "{{seed}}": str(seed
                     "{{width}}": str(w), "{{height}}": str(h), "{{length}}": str
            txt = txt.replace(k, v)
        return json.loads(txt)
    neg = "worst quality, inconsistent motion, blurry, jittery, distorted, waterm
    return {
        "1": {"class_type": "CheckpointLoaderSimple", "inputs": {"ckpt_name": LTX
        "2": {"class_type": "CLIPLoader", "inputs": {"clip_name": LTX_T5, "type":
        "3": {"class_type": "CLIPTextEncode", "inputs": {"text": prompt, "clip":
        "4": {"class_type": "CLIPTextEncode", "inputs": {"text": neg, "clip": ["2
        "5": {"class_type": "LTXVConditioning", "inputs": {
            "positive": ["3", 0], "negative": ["4", 0], "frame_rate": float(FPS)}
        "6": {"class_type": "EmptyLTXVLatentVideo", "inputs": {
            "width": w, "height": h, "length": frames, "batch_size": 1}},
        "7": {"class_type": "LTXVScheduler", "inputs": {
            "steps": 20, "max_shift": 2.05, "base_shift": 0.95,
```

```
            "stretch": True, "terminal": 0.1, "latent": ["6", 0]}},
        "8": {"class_type": "KSamplerSelect", "inputs": {"sampler_name": "euler"}
        "9": {"class_type": "SamplerCustom", "inputs": {
            "model": ["1", 0], "add_noise": True, "noise_seed": seed, "cfg": 3.0,
            "positive": ["5", 0], "negative": ["5", 1], "sampler": ["8", 0],
            "sigmas": ["7", 0], "latent_image": ["6", 0]}},
        "10": {"class_type": "VAEDecode", "inputs": {"samples": ["9", 0], "vae":
        "11": {"class_type": "SaveWEBM", "inputs": {
            "images": ["10", 0], "filename_prefix": "freealexya_vid",
            "codec": "vp9", "fps": float(FPS), "crf": 32.0}},
    }

# ---------- remplacement de personnage (Wan 2.1 VACE 1.3B + pose) ----------
VACE_UNET = os.getenv("VACE_UNET", "wan2.1_vace_1.3B_fp16.safetensors")
WAN_T5 = os.getenv("WAN_T5", "umt5_xxl_fp8_e4m3fn_scaled.safetensors")
WAN_VAE = os.getenv("WAN_VAE", "wan_2.1_vae.safetensors")

def fetch(url, dest):
    dest.write_bytes(requests.get(url, timeout=120).content)
    return dest

def comfy_upload(path):
    with open(path, "rb") as f:
        r = requests.post(f"{COMFY}/upload/image", files={"image": (path.name, f)
                          data={"overwrite": "true"}, timeout=120)
    r.raise_for_status()
    return r.json()["name"]

def probe(path):
    out = subprocess.check_output(["ffprobe", "-v", "error", "-select_streams", "
        "-show_entries", "stream=width,height", "-show_entries", "format=duration
        "-of", "json", str(path)])
    j = json.loads(out)
    return j["streams"][0]["width"], j["streams"][0]["height"], float(j["format"]

def prepare_replace(jid, source_url, character_url):
    """Télécharge, recadre le clip (16 fps, <= 3 s, 576x320 ou 320x576) et envoie
    src = fetch(source_url, REF / f"{jid}_src.mp4")
    sw, sh, dur = probe(src)
    w, h = (576, 320) if sw >= sh else (320, 576)
    secs = max(1, min(3, int(dur)))
    clip = REF / f"{jid}_clip.mp4"
    subprocess.check_call(["ffmpeg", "-y", "-loglevel", "error", "-i", str(src),
        "-r", "16", "-an", "-vf", f"scale={w}:{h}:force_original_aspect_ratio=inc
        str(clip)])
    ref = fetch(character_url, REF / f"{jid}_char.png")
```

```
    return {"video": comfy_upload(clip), "image": comfy_upload(ref),
            "w": w, "h": h, "length": 16 * secs + 1}

def replace_workflow(prompt, seed, x):
    neg = "blurry, low quality, deformed, extra limbs, watermark, static"
    return {
        "1": {"class_type": "UNETLoader", "inputs": {"unet_name": VACE_UNET, "wei
        "2": {"class_type": "CLIPLoader", "inputs": {"clip_name": WAN_T5, "type":
        "3": {"class_type": "VAELoader", "inputs": {"vae_name": WAN_VAE}},
        "4": {"class_type": "ModelSamplingSD3", "inputs": {"model": ["1", 0], "sh
        "5": {"class_type": "CLIPTextEncode", "inputs": {"text": prompt, "clip":
        "6": {"class_type": "CLIPTextEncode", "inputs": {"text": neg, "clip": ["2
        "7": {"class_type": "LoadVideo", "inputs": {"file": x["video"]}},
        "8": {"class_type": "GetVideoComponents", "inputs": {"video": ["7", 0]}},
        "9": {"class_type": "ImageFromBatch", "inputs": {
            "image": ["8", 0], "batch_index": 0, "length": x["length"]}},
        "10": {"class_type": "DWPreprocessor", "inputs": {
            "image": ["9", 0], "detect_hand": "enable", "detect_body": "enable",
            "detect_face": "enable", "resolution": 512,
            "bbox_detector": "yolox_l.onnx", "pose_estimator": "dw-ll_ucoco_384.o
        "11": {"class_type": "LoadImage", "inputs": {"image": x["image"]}},
        "12": {"class_type": "WanVaceToVideo", "inputs": {
            "positive": ["5", 0], "negative": ["6", 0], "vae": ["3", 0],
            "width": x["w"], "height": x["h"], "length": x["length"], "batch_size
            "strength": 1.0, "control_video": ["10", 0], "reference_image": ["11"
        "13": {"class_type": "KSampler", "inputs": {
            "model": ["4", 0], "positive": ["12", 0], "negative": ["12", 1],
            "latent_image": ["12", 2], "seed": seed, "steps": 20, "cfg": 6.0,
            "sampler_name": "uni_pc", "scheduler": "simple", "denoise": 1.0}},
        "14": {"class_type": "TrimVideoLatent", "inputs": {"samples": ["13", 0],
        "15": {"class_type": "VAEDecode", "inputs": {"samples": ["14", 0], "vae":
        "16": {"class_type": "SaveWEBM", "inputs": {
            "images": ["15", 0], "filename_prefix": "freealexya_replace",
            "codec": "vp9", "fps": 16.0, "crf": 32.0}},
    }

# ---------- exécution ----------
def run_comfy(wf):
    r = requests.post(f"{COMFY}/prompt", json={"prompt": wf, "client_id": str(uui
    r.raise_for_status()
    pid = r.json()["prompt_id"]
    while True:
        time.sleep(2)
        hist = requests.get(f"{COMFY}/history/{pid}", timeout=30).json()
        if pid in hist:
            outs = hist[pid]["outputs"]
```

```
            for node in outs.values():
                for key in ("images", "gifs", "videos"):
                    for f in node.get(key, []):
                        return f
            raise RuntimeError("Aucune sortie produite")

def worker(jid, kind, prompt, aspect, seed, frames, extra=None):
    try:
        set_job(jid, status="processing")
        if kind == "image":
            wf = image_workflow(prompt, aspect, seed)
        elif kind == "replace":
            wf = replace_workflow(prompt, seed, prepare_replace(jid, extra["sourc
        else:
            wf = video_workflow(prompt, aspect, seed, frames)
        f = run_comfy(wf)
        r = requests.get(f"{COMFY}/view", params={
            "filename": f["filename"], "subfolder": f.get("subfolder", ""),
            "type": f.get("type", "output")}, timeout=120)
        ext = Path(f["filename"]).suffix
        (OUT / f"{jid}{ext}").write_bytes(r.content)
        set_job(jid, status="completed", output_url=f"{PUBLIC}/files/out/{jid}{ex
    except Exception as e:
        set_job(jid, status="failed", error=str(e))

def submit(kind, prompt, aspect, seed, frames=33, extra=None):
    jid = str(uuid.uuid4())
    seed = seed if seed is not None else random.randint(0, 2**32 - 1)
    with lock:
        db.execute("INSERT INTO jobs VALUES(?,?,?,?,?,?,?,?)",
                   (jid, kind, prompt, json.dumps({"aspect": aspect, "seed": seed
                    "queued", None, None, time.time()))
        db.commit()
    threading.Thread(target=worker, args=(jid, kind, prompt, aspect, seed, frames
    return {"id": jid, "status": "queued", "cost_credits": 0}

# ---------- MCP ----------
mcp = FastMCP("FreeAlexya", host="0.0.0.0", port=PORT)

@mcp.tool()
def generate_image(prompt: str, aspect_ratio: str = "1:1", seed: int | None = Non
    """Génère une image (FLUX schnell via ComfyUI). Renvoie un id à suivre avec c
    return submit("image", prompt, aspect_ratio, seed)

@mcp.tool()
def generate_video(prompt: str, aspect_ratio: str = "16:9", duration: int = 2,
```

```
                    seed: int | None = None) -> dict:
    """Génère une courte vidéo (LTX-Video via ComfyUI). duration en secondes (1-4
    return submit("video", prompt, aspect_ratio, seed, frames=min(97, max(1, roun

@mcp.tool()
def generate_replace_video(source_video_url: str, character_image_url: str,
                           prompt: str = "a person, same movements, high quality"
                           seed: int | None = None) -> dict:
    """Remplace la personne d'un clip (max 3 s, recadré en 16 fps) par le personn
    Garde les mouvements (pose) ; le décor est regénéré. Wan 2.1 VACE 1.3B."""
    return submit("replace", prompt, "", seed, extra={"source": source_video_url,
                                                       "character": character_imag

@mcp.tool()
def check_generation(generation_id: str) -> dict:
    """Statut d'un job : queued / processing / completed / failed + output_url.""
    row = db.execute("SELECT id,kind,status,output_url,error FROM jobs WHERE id=?
                     (generation_id,)).fetchone()
    if not row:
        return {"error": "id inconnu"}
    return dict(zip(("id", "kind", "status", "output_url", "error"), row))

@mcp.tool()
def list_generations(limit: int = 20) -> list:
    """Dernières générations."""
    rows = db.execute("SELECT id,kind,prompt,status,output_url FROM jobs "
                      "ORDER BY created DESC LIMIT ?", (limit,)).fetchall()
    return [dict(zip(("id", "kind", "prompt", "status", "output_url"), r)) for r

@mcp.tool()
def upload_reference(source_url: str) -> dict:
    """Télécharge une image/vidéo de référence et renvoie une URL locale."""
    name = f"{uuid.uuid4()}{Path(source_url.split('?')[0]).suffix or '.bin'}"
    (REF / name).write_bytes(requests.get(source_url, timeout=60).content)
    return {"public_url": f"{PUBLIC}/files/ref/{name}"}

@mcp.tool()
def get_account() -> dict:
    """Pas de crédits : tout est illimité (limité par votre GPU)."""
    return {"credits": "unlimited"}

@mcp.custom_route("/files/{folder}/{name}", methods=["GET"])
async def files(request):
    folder, name = request.path_params["folder"], Path(request.path_params["name"
    base = {"out": OUT, "ref": REF}.get(folder)
    p = base / name if base else None
```

```
    return FileResponse(p) if p and p.exists() else PlainTextResponse("404", stat

if __name__ == "__main__":
    mcp.run(transport="streamable-http")  # endpoint : http://localhost:8000/mcp
```